Adevar Labs

Prepared for:
Altura

December 17, 2025

Altura NavVault Audit Report

Version: after-fix report

Table of Contents

1. Executive Summary

About Altura

The protocol is centered around a Net Asset Value (NAV) based investment vault implemented in the NavVault contract, which adheres to the ERC-4626 standard. The core mechanism of the vault relies on an external NavOracle to determine the share price (pricePerShare) against the underlying asset, thus calculating total assets and conversion rates.

The NavVault extends the standard ERC-4626 with custom features:

Share price determined by the NavOracle based on the performance of external strategies in the Hyperliquid ecosystem rather than totalAssets() / totalShares().

An epoch-based withdrawal queue that allows users to queue their withdrawal requests, with shares becoming claimable only after the end of the current epoch.

A referral system that allows users to bind a referrer address upon their first deposit or mint, used for attribution purposes.

Both the vault and the oracle utilize OpenZeppelin's AccessControl and Pausable features for administrative control and emergency shutdown.

Audit Summary

Number of Findings: 8 total

- Critical: 0
- High: 0
- Medium: 2
- Low: 6

Number of Enhancement Opportunities: 3

Risk Profile

The following table summarizes the distribution of identified vulnerabilities by risk level:

Risk Level	Count	Fixed	Acknowledged
Critical	0	0	0
High	0	0	0
Medium	2	2	0
Low	6	5	1

Overall Security Posture

After Fix Review

The fix review confirms that all Medium severity findings were resolved. Additionally, one Low severity issue (USDT fee activation) was acknowledged, while all remaining issues were fixed. Overall, the fixes address predictable oracle price arbitrage, rounding errors, and the introduction of a safe pausing mechanism. Finally, it must be noted that the protocol maintains centralized control over fund movement, as the moveAssets function allows the operator to move funds to a configurable address.

Before Fix Review

The Altura team has established a good foundation by utilizing the OpenZeppelin contracts for fundamental components, including ERC4626, AccessControl, Pausable, and ReentrancyGuard. The NavOracle contract shows good security practice by enforcing a maximum price-per-share (PPS) movement limit (maxPpsMoveBps) and an explicit staleness check. The NavOracle

Our review uncovered several security findings that require attention:

Risk of Key Compromise: The setOracle function lacks a time-lock mechanism and the moveAssets function allows the operator to transfer funds to any arbitrary address.

Predictable Oracle Arbitrage: The oracle's predictable update schedule enables a low-risk, near-instant arbitrage opportunity for sophisticated users, allowing them to extract yield at the expense of other depositors.

Incorrect ERC4626 Rounding: The incorrect rounding direction in the core share conversion functions, which omits the necessary ERC4626 rounding rules, enables value extraction by an attacker and dilution of existing shareholder value.

The implementation of custom logic, such as the withdrawal queue and referral system, also contains minor flaws, including a missing whenNotPaused modifier on cancelWithdrawal and an incomplete ERC20 allowance consumption logic in queueWithdrawal.

Launch Recommendations

After Fix Review

All mandatory before-launch steps have been implemented. Following the fix review, we recommend implementing the suggested steps in "Post-Launch Monitoring" to ensure anomalous on-chain activity is detected early and a clear plan is established to address it.

Before Fix Review

To ensure a smooth and secure launch, we offer the following recommendations:

I. Mandatory Before Launch:

Fix the "Medium" severity issues. Specifically:

Implement a mechanism (e.g., a fee or fixed withdrawal delay) to mitigate the Predictable Oracle Arbitrage opportunity.

Resolve the Incorrect Rounding Direction in _convertToShares and _convertToAssets by following the OpenZeppelin ERC4626 standard and including the rounding parameter.

II. Strongly Recommended:

Fix all Low severity issues:

Add the whenNotPaused modifier to the cancelWithdrawal function.

Fix the incomplete implementation in queueWithdrawal by consuming the user's ERC20 allowance via transferFrom.

Implement user-side slippage checks on all core vault functions (deposit, mint, withdraw, redeem).

Implement a Timelock mechanism for the setOracle function to prevent immediate compromise of the oracle dependency.

Restrict the moveAssets function to only transfer assets to a pre-configured and time-locked destination address.

Expand the test suite to include adversarial scenarios, boundary conditions, and state manipulation tests to validate security properties and system invariants more thoroughly.

III. Post-Launch Monitoring:

Monitor key admin roles for unauthorized transactions or changes in configuration.

Implement alerting for sudden, large price movements in the oracle feed or unexpected fund movements via moveAssets.

Establish clear procedures for private key security management and post-hack management.

Audit Scope

Repository: https://github.com/AlturaTrade/contracts/

Before-fix review commit hash: af5d6b811d3b6e7f51be97e670a91d41e19fcf7f

After-fix review commit hash: c504e2e1d4234275be36933d8502399639aab475

Files / Modules in Scope:

NavOracle.sol

NavVault.sol

2. Assumptions and Considerations

Audit Assumptions

I. The only supported vault asset will be USDT.

II. Withdrawal fees will be set such that oracle update arbitrage becomes unprofitable.

Centralization Risks

I. DEFAULT_ADMIN_ROLE, OPERATOR_ROLE, GUARDIAN_ROLE, REPORTER_ROLE are trusted and protocol-owned addresses.

II. DEFAULT_ADMIN_ROLE and OPERATOR_ROLE have the ability to move the funds out of the vault to an arbitrary defined address.

III. PPS (Price Per Share) is computed externally and users must trust the REPORTER_ROLE for delivering accurate prices.

3. Severity Definitions

Each issue identified in this report is assigned a severity level based on two dimensions: **Impact** and **Likelihood**. These dimensions help project our team's understanding of both the potential consequences of a vulnerability and how likely a vulnerability is to be discovered and exploited in the real world.

Note: Enhancements represent non-blocking improvements typically usability, observability, or defense-in-depth tweaks that do not pose an immediate asset risk but would improve the product s reliability and/or user experience if implemented.

Impact

Impact reflects the potential consequences of the issue particularly on **project funds**, **user funds**, and the **availability or integrity** of the protocol.

High Impact: Successful exploitation could result in a complete loss of user or protocol funds, disruption of core protocol functionality, or permanent loss of control over critical components.

Medium Impact: Exploitation could cause significant disruption or partial loss of funds, but not a total compromise. May impact some users or non-core functionality.

Low Impact: The issue has minor or negligible consequences. It may affect edge cases, expose metadata, or degrade performance slightly without putting funds or core logic at serious risk.

Likelihood

Likelihood reflects how easy a vulnerability is to discover and exploit by an attacker, as well as how economically attractive the exploit is to an attacker.

High Likelihood: The vulnerability is trivially exploitable. This means it can be exploited by a wide range of actors without privileged access rights, with minimal capital requirements and low financial risks.

Medium Likelihood: This type of vulnerability can be found and exploited with moderate effort. It might require a significant capital investment, but with manageable financial risk.

Low Likelihood: Exploitation of these vulnerabilities is often technically unfeasible or requires highly specialized conditions. They may require extraordinary effort or a significant financial risk for an attacker, with a high chance of failure and minimal potential return.

Severity Classification Matrix

By combining **Impact** and **Likelihood**, we assign a severity level using the matrix below:

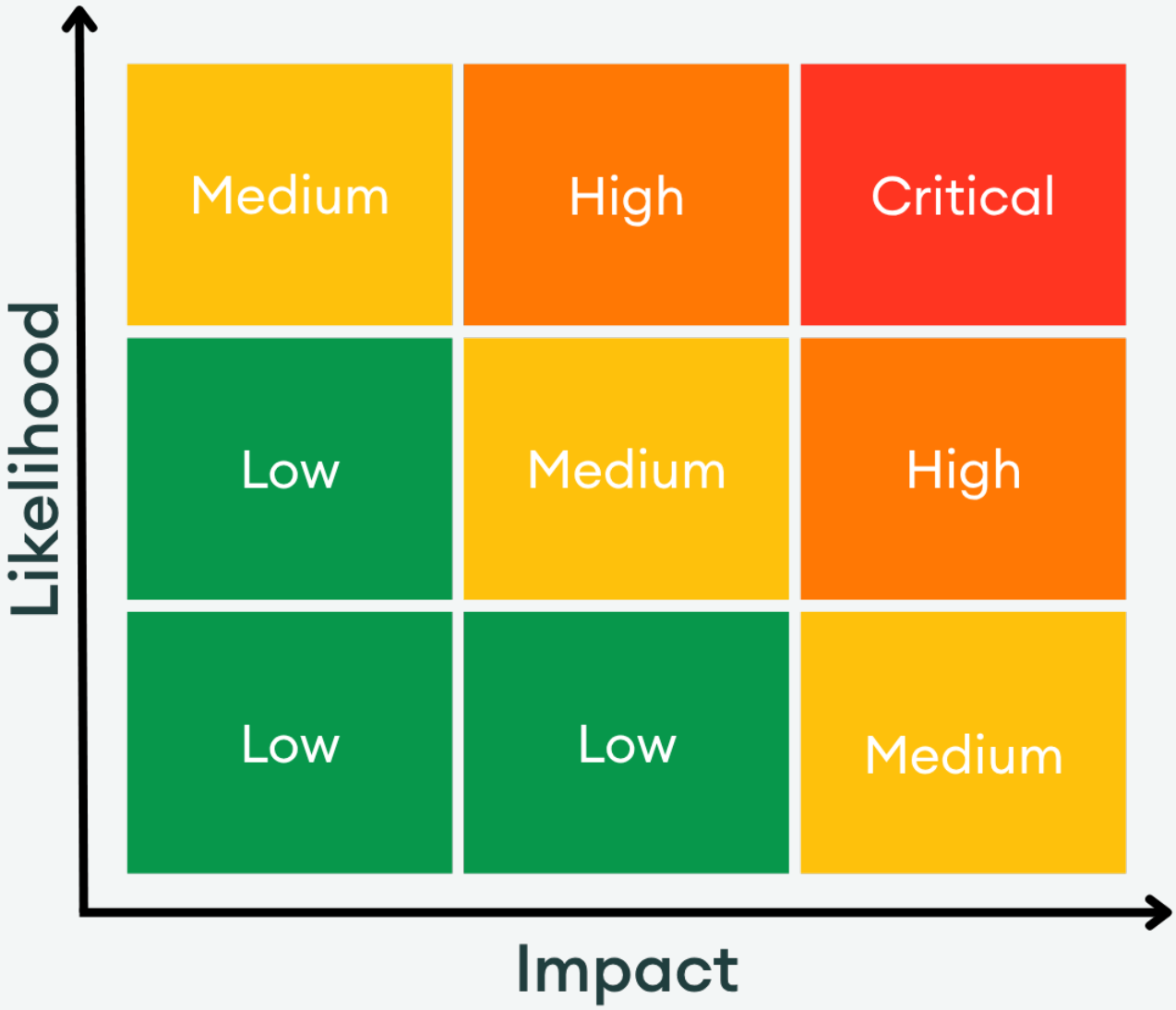
Critical: High impact + high likelihood (e.g. a bug that could allow anyone to drain a substansial amount of protocol funds with minimal effort)

High: High impact with medium likelihood, or medium impact with high likelihood

Medium: Moderate impact and/or discoverability

Low: Minimal impact or unlikely to be exploited

This structured approach helps teams prioritize fixes and mitigate the most dangerous threats first.



Severity Matrix

4. Findings

M01: Incorrect rounding direction in share and asset conversion leads to dilution of shareholder value

Status: Resolved

Impact:

Likelihood:

Severity: Medium

Location: https://github.com/AdevarLabs/altura_code/blob/main/contracts/NavVault.sol#L189

>_ NavVault.sol — SOLIDITY

```
187:     }
188:
189:     function _convertToShares(uint256 assets, Math.Rounding /*rounding*/)
190:         internal
191:         view
```

Description:

The NavVault contract implements the ERC4626 standard and overrides the virtual functions _convertToShares and _convertToAssets. For comparison, we show the OpenZeppelin implementation of these functions followed by the NavVault implementation.

OpenZeppelin

SOLIDITY

```
/**
 *@dev Internal conversion function (from asset to share) with support for rounding direction
 */
function _convertToShares(uint256 assets, Math.Rounding rounding) internal view virtual
returns (uint256) {
    return assets.mulDiv(totalSupply() + 10 ** _decimalsOffset(), totalAssets() + 1, rounding)
    ;
}

/**
 *@dev Internal conversion function (from share to asset) with support for rounding direction
 */
function _convertToAssets(uint256 shares, Math.Rounding rounding) internal view virtual
returns (uint256) {
    return shares.mulDiv(totalAssets() + 1, totalSupply() + 10 ** _decimalsOffset(), rounding)
    ;
}
```

NavVault

```solidity
function _convertToShares(uint256 assets, Math.Rounding /*rounding*/)
    internal
    view
    override
    returns (uint256)
{
    if (assets == 0) return 0;
    (uint256 pps, uint256 scale) = _oraclePpsScaled();
    return assets * scale / pps;
}

function _convertToAssets(uint256 shares, Math.Rounding /*rounding*/)
    internal
    view
    override
    returns (uint256)
{
    if (shares == 0) return 0;
    (uint256 pps, uint256 scale) = _oraclePpsScaled();
    return shares * pps / scale;
}
```

The key detail to note is that the NavVault omits the rounding parameter and will round down in every case. The rounding parameter is a crucial component of ERC4626 logic that was added to always round in favor of the vault and existing shareholders.

To demonstrate this, we see that in the OpenZeppelin preview functions, different rounding is used based on the direction that favors the vault.

```solidity
/// @inheritdoc IERC4626
function previewDeposit(uint256 assets) public view virtual returns (uint256) {
    return _convertToShares(assets, Math.Rounding.Floor);
}

/// @inheritdoc IERC4626
function previewMint(uint256 shares) public view virtual returns (uint256) {
    return _convertToAssets(shares, Math.Rounding.Ceil);
}

/// @inheritdoc IERC4626
function previewWithdraw(uint256 assets) public view virtual returns (uint256) {
    return _convertToShares(assets, Math.Rounding.Ceil);
}

/// @inheritdoc IERC4626
function previewRedeem(uint256 shares) public view virtual returns (uint256) {
    return _convertToAssets(shares, Math.Rounding.Floor);
}
```

Notably, in the current NavVault implementation, the rounding direction in previewMint and previewWithdraw will be floor instead of ceil, meaning that a user will:

1. Pay less assets to mint the same amount of shares
2. Burn less shares to withdraw the same amount of assets

When combined, these two effects provide the opportunity for an attacker to perform value extraction and dilute existing shareholder value.

Recommendation:

Follow the correct OpenZeppelin ERC4626 standard and include the rounding parameter in the conversion calculations.

DIFF

```
- function _convertToShares(uint256 assets, Math.Rounding /*rounding*/)
+ function _convertToShares(uint256 assets, Math.Rounding rounding)
    internal
    view
    override
    returns (uint256)
{
    if (assets == 0) return 0;
    (uint256 pps, uint256 scale) = _oraclePpsScaled();
-    return assets * scale / pps;
+   return assets.mulDiv(scale, pps, rounding);
}

- function _convertToAssets(uint256 shares, Math.Rounding /*rounding*/)
+ function _convertToAssets(uint256 shares, Math.Rounding rounding)
    internal
    view
    override
    returns (uint256)
{
    if (shares == 0) return 0;
    (uint256 pps, uint256 scale) = _oraclePpsScaled();
-   return shares * pps / scale;
+  return shares.mulDiv(pps, scale, rounding)
}
```

Proof of Concept:

Add the following to Vault.t.ts:

TS

```
it("Demonstrates rounding error and value extraction", async () => {
    // Setup
    const { user, usdc, reporter, oracle, vault } = await deployFixture();

    // Update PPS to 1_000_001
    const ts = (await latestTs()) + 10;
    await oracle.connect(reporter).reportNav(ethers.parseEther("1.000001"), ts);

    const shares_to_mint = 999_999;
    const assets_to_withdraw = 1_000_000;

    // 1. Convert to Assets Rounding Error (Minting)
    // Formula: assets = shares * pps_e6 / scale
    // Calculate assets needed for shares_to_mint (999,999 shares):
    // assets = 999_999 * 1_000_001 / 1_000_000 = ~999,999.9999999 assets (floor to 999,999)
    const expected_assets_paid = 999_999;
    const actual_assets_paid = await vault.previewMint(shares_to_mint);
```

... continued

```ts
    expect(actual_assets_paid).to.equal(expected_assets_paid); // Rounding down 1 share

    // 2. Convert to Shares Rounding Error (Withdraw)
    // Formula: shares = assets * scale / pps_e6
    // Now, calculate the shares burned to withdraw those assets (1_000_000 assets)
    // shares = 1_000_000 * 1_000_000  / 1_000_001 = 999,999.000001 shares (floor to 999,999)
    const expected_shares_burned = 999_999n;
    const actual_shares_burned = await vault.previewWithdraw(assets_to_withdraw);
    expect(actual_shares_burned).to.equal(expected_shares_burned); // Rounding down 1 asset

    // This rounding error allows the user to mint the required shares for 999,999 assets and
    later redeem them to withdraw 1,000,000 assets, resulting in a net gain.
  });
```

Developer Response:

Implemented the recommended fix to use the rounding parameter in _convertToShares and _convertToAssets.

M02: Predictable oracle updates enable low-risk arbitrage

Status: Resolved

Impact: Low | Medium | High

Likelihood: Low | Medium | High

Severity: Medium

Location:
https://github.com/AdevarLabs/altura_code/blob/main/contracts/NavVault.sol#L320-L330

>_ NavVault.sol SOLIDITY

```
318:     }
319:
320:     function withdraw(uint256 assets, address receiver, address owner)
321:         public
322:         override
323:         whenNotPaused
324:         nonReentrant
325:         returns (uint256 sharesBurned)
326:     {
327:         sharesBurned = super.withdraw(assets, receiver, owner);
328:         cumulativeWithdrawals += assets;
329:         emit FlowCountersUpdated(cumulativeDeposits, cumulativeWithdrawals);
330:     }
331:
332:     function redeem(uint256 shares, address receiver, address owner)
```

Description:

The vault's withdrawal system allows users to either directly withdraw assets from the vault by redeeming their shares by calling withdraw or redeem, or if not enough assets are available at that time, they can queue a withdrawal by calling queueWithdrawal. That requested amount is expected to be available in the next epoch (every 1 day or 86400 seconds), allowing users to finally claim their assets by calling claimWithdrawal.

The issue is that the system allows users to exploit predictable oracle updates with minimal to zero risk exposure.
This is a common issue that is found in vaults using price oracles, see Angle's article on the topic.

This arises because the vault uses an oracle that is supposed to be updated at fixed intervals (5 minutes). This oracle is updated by a trusted role (REPORTER_ROLE) through the reportNav function where the new price is pushed based on the available funds in the vault's underlying strategies. Because Altura's vault goal is to generate profit (positive APY) using well-designed trading strategies, it is expected that PPS will more frequently increase than decrease.
Because the oracle update time and value (called PPS) are predictable, a user can deposit assets right before a new price is pushed in order to generate a near-instant profit:

1. User deposits before oracle update (e.g., PPS = 1 and user deposits 1000 USDC)
2. PPS is updated to 1.01
3. User calls withdraw or redeem in the next block, limiting their risk exposure to 1 block time (1 second for fast blocks on Hyperliquid)

4. User generates 10 USDC profit in 2 seconds

This issue, while most difficult and likely to be exploited, is also present with the claimWithdrawal function. The function requires a user to first call queueWithdrawal and wait for the next epoch to claim its assets. There is then a (small and less frequent) window where the next epoch is aligned with the oracle update.

Such arbitrage is done to the disadvantage of the other depositors who see part of the generated yield being extracted by arbitragers that aren't actively participating in the strategy.
Because the strategy is expected to have a steady yield, this creates an unfair advantage for arbitragers.

Proof of Concept:

We can try to estimate the possible arbitrage opportunity every 5 minutes by going with an arbitrary 20% APY.
This would give us an expected price increase of 20%^(1/12) = 1.3%/month. Then, we can compute the expected increase by oracle update (i.e., every 5 minutes), which would give us a 0.00015%/oracle-update increase in price.
With 10M USDC available, and considering close to $0.01 transaction fees (as the protocol is deployed on Hyperliquid), this allows earning ~$1.5 every 5 minutes, or $432 every day in the best scenario (price always increases).

Recommendation:

To prevent value extraction from a vault, different methods could be used:
1. Implementation of a deposit queue where deposits are executed in a two-step process: queue deposit -> confirm deposit where the deposit confirmation must be executed at least one block after being queued. The shares should be minted during the second step to ensure shares are minted after the oracle is updated.
2. Implementation of a withdrawal fee to reduce arbitrage profitability. The fee should exceed typical oracle price movements, making the attack economically unviable. The fee could be applied only on early withdrawals to affect only arbitragers.

Developer Response:

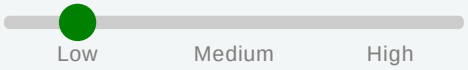
A configurable withdrawal fee, capped at 2%, has been implemented.
This approach allows the protocol to effectively eliminate oracle arbitrage opportunities by setting fees high enough to make such operations unprofitable (see proof of concept for an estimate of required fees), while still remaining low enough to minimize user impact.

L01: Unhandled fee-on-transfer behavior will cause accounting discrepancies if USDT enables transfer fees

Status: Acknowledged

Impact:

Likelihood:



Severity: Low

Location:
https://github.com/AdevarLabs/altura_code/blob/main/contracts/NavVault.sol#L295-L306

>_ NavVault.sol — SOLIDITY

```
293:     }
294:
295:     // ===== Mutating ERC4626 (standard) =====
296:     function deposit(uint256 assets, address receiver)
297:         public
298:         override
299:         whenNotPaused
300:         nonReentrant
301:         returns (uint256 shares)
302:     {
303:         shares = super.deposit(assets, receiver);
304:         cumulativeDeposits += assets;
305:         emit FlowCountersUpdated(cumulativeDeposits, cumulativeWithdrawals);
306:     }
307:
308:     function mint(uint256 shares, address receiver)
```

Description:

The NavVault contract is designed to accept ERC20 assets including USDT which has a dormant fee-on-transfer mechanism. While USDT currently does not charge transfer fees, the token contract contains the capability to enable fees at any time.
The contract assumes that the amount of tokens specified in the function call is exactly what the contract receives. The protocol uses safeTransferFrom() to pull tokens and immediately updates accounting based on the requested amount rather than measuring the actual tokens received:

SOLIDITY

```
// ERC4626 deposit function
    function _deposit(address caller, address receiver, uint256 assets, uint256 shares) intern
    al virtual {
        SafeERC20.safeTransferFrom(IERC20(asset()), caller, address(this), assets);
        _mint(receiver, shares);

        emit Deposit(caller, receiver, assets, shares);
    }
```

If USDT enables its fee-on-transfer mechanism (e.g., 0.1% per transfer), the contract would receive less than the stated assets amount, but the accounting would be based on the pre-fee amount.

Recommendation:

Update the NavVault::deposit function to measure the actual token amount received by checking the balance before and after transfers:

SOLIDITY

```
function deposit(uint256 assets, address receiver)
      public
      override
      whenNotPaused
      nonReentrant
      returns (uint256 shares)
  {
      uint256 maxAssets = maxDeposit(receiver);
      if (assets > maxAssets) {
          revert ERC4626ExceededMaxDeposit(receiver, assets, maxAssets);
      }

      IERC20 assetToken = IERC20(address(asset()));
      uint256 balanceBefore = assetToken.balanceOf(address(this));

      // Pull tokens from sender
      assetToken.safeTransferFrom(msg.sender, address(this), assets);

      // Measure actual amount received (handles fee-on-transfer)
      uint256 balanceAfter = assetToken.balanceOf(address(this));
      uint256 actualReceived = balanceAfter - balanceBefore;

      // Mint shares based on actual received amount
      shares = previewDeposit(actualReceived);
      _mint(receiver, shares);

      cumulativeDeposits += actualReceived;
      emit FlowCountersUpdated(cumulativeDeposits, cumulativeWithdrawals);
      emit Deposit(msg.sender, receiver, actualReceived, shares);
  }
```

The same logic should be followed for the NavVault::mint and fundLiquidity functions.

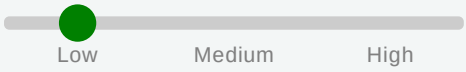
Developer Response:

Acknowledged, the risk of USDT implementing fees is considered sufficiently low.

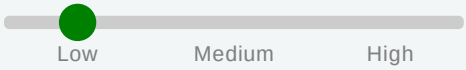
L02: Lack of slippage checks exposes users to unexpected oracle price updates

Status: Resolved

Impact:



Likelihood:



Severity: Low

Location: https://github.com/AdevarLabs/altura_code/blob/main/contracts/NavVault.sol#L296

>_ NavVault.sol SOLIDITY

```
294:
295:     // ===== Mutating ERC4626 (standard) =====
296:     function deposit(uint256 assets, address receiver)
297:         public
298:         override
```

Description:

The core vault functions (mint, redeem, withdraw, deposit) are missing slippage checks. Due to the nature of the oracle supplied share price, the users of the protocol are not vulnerable to getting front-runned by other users' actions. However, the change in price of shares set by the oracle within the same block as the user's vault operation could lead to unexpected amounts of shares/assets being used.

This has been already partially mitigated by the maxPpsMoveBps parameter, which is checked when the oracle sets a new PPS. However, a user-side slippage check would explicitly allow the user to specify their loss tolerance.

Recommendation:

Add slippage checks to all core vault functions and revert the transactions if these are not met.

Developer Response:

Added withCheck variants for each of the four main vault operations with slippage checks.

L03: Missing whenNotPaused modifier permits user action during paused state

Status: Resolved

Impact: Low Medium High

Likelihood: Low Medium High

Severity: Low

Location: https://github.com/AdevarLabs/altura_code/blob/main/contracts/NavVault.sol#L408

>_ NavVault.sol SOLIDITY

```
406:     }
407:
408:     function cancelWithdrawal(uint256 id) external nonReentrant {
409:         WithdrawalRequest storage r = requests[id];
410:         if (r.owner != msg.sender) revert NotOwner();
```

Description:

The protocol inherits the Pausable contract, allowing the guardian role to pause all vault operations. However, the cancelWithdrawal function is missing the whenNotPaused modifier, allowing users to transfer escrowed shares from the vault back to themselves. This affects protocol economics during the paused state and should not be allowed.

Recommendation:

Add the whenNotPaused modifier to the cancelWithdrawal function.

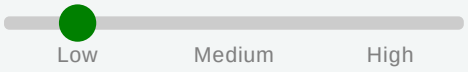
Developer Response:

Added the whenNotPaused modifier to the cancelWithdrawal function.

L04: Incomplete implementation when transferring shares from a user in queueWithdrawal

Status: Resolved

Impact:



Likelihood:

Severity: Low

Location:
https://github.com/AdevarLabs/altura_code/blob/main/contracts/NavVault.sol#L350-L362

>_ NavVault.sol — SOLIDITY

```
348:     }
349:
350:     function queueWithdrawal(uint256 shares, address receiver)
351:         external
352:         whenNotPaused
353:         nonReentrant
354:         returns (uint256 id)
355:     {
356:         if (shares == 0) revert ZeroAmount();
357:         if (receiver == address(0)) receiver = msg.sender;
358:
359:         uint256 allowance_ = allowance(msg.sender, address(this));
360:         if (allowance_ < shares) revert InsufficientApproval();
361:
362:         _transfer(msg.sender, address(this), shares);
363:
364:         uint64 nowTs = uint64(block.timestamp);
```

Description:

queueWithdrawal allows users to request a withdrawal from the vault by escrowing the shares into the vault.
The function first checks for sufficient allowance from the caller, then uses the internal _transfer function to transfer the shares from the user's balance.
The issue is that while the shares are transferred, the allowance is not consumed.

According to the ERC20 specification, when tokens are transferred from an address it must consume that allowance during the transfer.

Deviating from the ERC20 specification breaks composability with other protocols and tools that rely on standard allowance behavior, leading to incorrect accounting for these protocols, failed integrations, and confusing users.

Recommendation:

Replace the current transfer mechanism by a transferFrom call:

DIFF

```
function queueWithdrawal(uint256 shares, address receiver)
```

... continued DIFF

```
    external
    whenNotPaused
    nonReentrant
    returns (uint256 id)
{
    if (shares == 0) revert ZeroAmount();
    if (receiver == address(0)) receiver = msg.sender;

+   transferFrom(msg.sender, address(this), shares);
-   uint256 allowance_ = allowance(msg.sender, address(this));
-   if (allowance_ < shares) revert InsufficientApproval();

-   _transfer(msg.sender, address(this), shares);

    // ... request registration logic ...

    emit WithdrawalQueued(id, msg.sender, receiver, shares, claimableAt);
}
```

Developer Response:

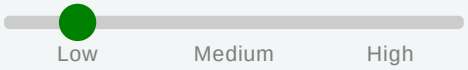
Allowance is now consumed when the vault is transferring shares from the user's balance in queueWithdrawal.

L05: Lack of a timelock mechanism in setOracle makes the protocol more vulnerable to private key compromise

Status: Resolved

Impact:

Likelihood:



Severity: Low

Location:
https://github.com/AdevarLabs/altura_code/blob/main/contracts/NavVault.sol#L133-L136

>_ NavVault.sol — SOLIDITY

```
131:
132:     // ===== Admin / Ops =====
133:     function setOracle(INavOracle oracle_) external onlyRole(DEFAULT_ADMIN_ROLE) {
134:         if (address(oracle_) == address(0)) revert BadAddress();
135:         navOracle = oracle_;
136:     }
137:
138:     function setMaxAllowedStaleness(uint256 secs) external onlyRole(OPERATOR_ROLE) {
```

Description:

The setOracle function allows the admin to immediately change the oracle contract address without any timelock delay. An attacker who compromises the DEFAULT_ADMIN_ROLE's key can instantly switch to a malicious oracle contract that reports fraudulent NAV prices, enabling immediate theft through share price manipulation.

This would allow the attacker to withdraw all available assets with any number of shares.

As most of the funds are expected to be outside of the vault in strategies, this lowers the amount of tokens that can be stolen.

Proof of Concept:

1. Attacker holds 1 vault share
2. Deploy oracle reporting price such as 1 share = total supply
3. Call setOracle with the malicious oracle
4. Redeem share and drain vault entirely

Recommendation:

Implement a timelock mechanism for oracle updates, allowing the protocol to be paused if any malicious oracle changes happens.

Example of implementation:

SOLIDITY

```
uint256 public constant ORACLE_UPDATE_DELAY = 2 days;
  INavOracle public pendingOracle;
  uint256 public oracleUpdateTimestamp;
```

... continued

```solidity
function setOracle(INavOracle oracle_) external onlyRole(DEFAULT_ADMIN_ROLE) {
    if (address(oracle_) == address(0)) revert BadAddress();
    pendingOracle = oracle_;
    oracleUpdateTimestamp = block.timestamp + ORACLE_UPDATE_DELAY;
    emit OracleUpdateInitiated(address(oracle_), oracleUpdateTimestamp);
}

function finalizeOracleUpdate() external onlyRole(DEFAULT_ADMIN_ROLE) {
    if (block.timestamp < oracleUpdateTimestamp) revert TimelockNotExpired();
    navOracle = pendingOracle;
    pendingOracle = INavOracle(address(0));
    emit OracleUpdated(address(navOracle));
}
```

Developer Response:

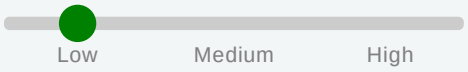
Fixed according to the recommendation.

L06: moveAssets function should transfer assets to a pre-configured address

Status: Resolved

Impact:

Likelihood:



Severity: Low

Location:
https://github.com/AdevarLabs/altura_code/blob/main/contracts/NavVault.sol#L422-L426

>_ NavVault.sol SOLIDITY

```
420:
421:     // ===== Liquidity Ops =====
422:     function moveAssets(address to, uint256 assets_) external whenNotPaused onlyRole(O
         PERATOR_ROLE) nonReentrant {
423:         if (to == address(0)) revert BadAddress();
424:         if (assets_ == 0) revert ZeroAmount();
425:         IERC20(address(asset())).safeTransfer(to, assets_);
426:     }
427:
428:     function fundLiquidity(uint256 assets_) external whenNotPaused nonReentrant {
```

Description:

The moveAssets function allows the OPERATOR_ROLE to transfer any amount of vault assets to any arbitrary address provided as a parameter. While the operator is intended to be a trusted role, this design creates an unnecessary attack vector if the operator's private key is compromised.

Currently, the function accepts a to parameter that can be any non-zero address, allowing a malicious actor to drain all available vault funds to wallets they control.

As most of the funds are expected to be outside of the vault in strategies, this lowers the amount of tokens that can be stolen.

Recommendation:

Restrict the moveAssets function to only transfer assets to a pre-configured destination address (or a whitelisted set of addresses).

As an additional layer of security, ensure that updating the destination address is done in two steps and is time-locked:
1. Proposing a new destination address
2. Executing the proposal once the time lock has passed

Implementing a time-lock delay (e.g., 2 days) before destination changes take effect gives time to detect and respond to malicious changes.

Developer Response:

The moveAssets function can now only transfer assets to a pre-configured address liquidityRecipient. The liquidityRecipient parameter is set by the DEFAULT_ADMIN_ROLE. Instead of a time lock, fund transfers require control over two separate roles: the Admin role to update the destination address and the Operator role to call moveAssets.

5. Enhancement Opportunities

E01: Removing redundant referrer checks will improve readability and gas consumption

Location:
https://github.com/AdevarLabs/altura_code/blob/main/contracts/NavVault.sol#L245-L252

>_ NavVault.sol SOLIDITY

```
243:         returns (uint256 assetsRequired)
244:     {
245:         if (referrer != address(0)) {
246:             if (referrer == receiver) revert InvalidReferrer();
247:
248:             if (referrerOf[receiver] == address(0)) {
249:                 if (msg.sender != receiver) revert InvalidReferrer();
250:                 _bindReferrerOnce(receiver, referrer);
251:             }
252:         }
253:
254:         assetsRequired = super.mint(shares, receiver);
```

Description:

The vault uses a referral system that performs several checks to ensure the validity of a referrer address. These checks are duplicated across mint/_bindReferrerOnce and deposit/_bindReferrerOnce.

mint/deposit checks

SOLIDITY

```
if (referrer != address(0)) { //Check 1)
      if (referrer == receiver) revert InvalidReferrer(); //Check 2)

      if (referrerOf[receiver] == address(0)) { //Check 3)
          if (msg.sender != receiver) revert InvalidReferrer();
          _bindReferrerOnce(receiver, referrer);
      }
 }
```

_bindReferrerOnce checks

SOLIDITY

```
if (referrerOf[user] != address(0)) revert ReferrerAlreadySet();  //Check 3) duplicated
if (referrer == address(0) || referrer == user) revert InvalidReferrer(); // Check 1) and Chec
    k 2) duplicated
referrerOf[user] = referrer;
emit ReferrerSet(user, referrer);
```

Potential Benefit:

Removing the redundant checks will offer gas savings and improved readability

Recommendation:

Remove the redundant checks in mint/deposit

DIFF

```
- if (referrer != address(0)) {
-     if (referrer == receiver) revert InvalidReferrer();

-     if (referrerOf[receiver] == address(0)) {
          if (msg.sender != receiver) revert InvalidReferrer();
          _bindReferrerOnce(receiver, referrer);
-     }
- }
```

E02: Configuration and liquidity management functions should emit events to improve visibility

Location: https://github.com/AdevarLabs/altura_code/blob/main/contracts/NavVault.sol#L135

>_ NavVault.sol — SOLIDITY

```
133:     function setOracle(INavOracle oracle_) external onlyRole(DEFAULT_ADMIN_ROLE) {
134:         if (address(oracle_) == address(0)) revert BadAddress();
135:         navOracle = oracle_;
136:     }
137:
```

Description:

Some functions in the Vault contract that make key configuration changes are missing event emissions. These include:

setOracle

```solidity
function setOracle(INavOracle oracle_) external onlyRole(DEFAULT_ADMIN_ROLE) {
        if (address(oracle_) == address(0)) revert BadAddress();
        navOracle = oracle_;
    }
```

setMaxAllowedStaleness

```solidity
function setMaxAllowedStaleness(uint256 secs) external onlyRole(OPERATOR_ROLE) {
        if (secs == 0) revert ZeroAmount();
        uint256 oracleCap = navOracle.maxOracleStaleness();
        if (secs > oracleCap) revert OracleStale();
        maxAllowedStaleness = secs;
    }
```

pause/unpause

```solidity
function pause() external onlyRole(GUARDIAN_ROLE) { _pause(); }
function unpause() external onlyRole(GUARDIAN_ROLE) { _unpause(); }
```

moveAssets

```solidity
function moveAssets(address to, uint256 assets_) external whenNotPaused onlyRole(OPERATOR_ROLE
    ) nonReentrant {
    if (to == address(0)) revert BadAddress();
    if (assets_ == 0) revert ZeroAmount();
    IERC20(address(asset())).safeTransfer(to, assets_);
}
```

fundLiquidity

```solidity
function fundLiquidity(uint256 assets_) external whenNotPaused nonReentrant {
    if (assets_ == 0) revert ZeroAmount();
    IERC20(address(asset())).safeTransferFrom(msg.sender, address(this), assets_);
}
```

Potential Benefit:

Improving visibility towards users and other integrators of the protocol such as indexers.

Recommendation:

Add event emissions at the end of each function.

E03: Adding timestamp check will prevent admin errors

Location: https://github.com/AdevarLabs/altura_code/blob/main/contracts/NavOracle.sol#L93

>_ NavOracle.sol SOLIDITY

```
91:     function reportNav(uint256 pps1e18, uint256 ts) external onlyRole(REPORTER_ROLE) w
        henNotPaused {
92:         if (pps1e18 == 0 || ts == 0) revert ZeroValue();
93:         if (ts < block.timestamp - _maxOracleStaleness) revert StaleTimestamp();
94:
95:         if (maxPpsMoveBps > 0) {
```

Description:

In the reportNav function, there is a check for staleness of the oracle timestamp but no check for the oracle timestamp being in the future.

Potential Benefit:

Prevent admin error of setting an oracle price in the future.

Recommendation:

Add the following check:

DIFF

```
function reportNav(uint256 pps1e18, uint256 ts) external onlyRole(REPORTER_ROLE) whenNotPaused
     {
        if (pps1e18 == 0 || ts == 0) revert ZeroValue();
        if (ts < block.timestamp - _maxOracleStaleness) revert StaleTimestamp();
+       if (ts > block.timestamp) revert FutureTimestamp();
        if (maxPpsMoveBps > 0) {
            uint256 prev = lastNav.pps1e18;
            if (prev > 0) {
                uint256 diff = prev > pps1e18 ? prev - pps1e18 : pps1e18 - prev;
                if (diff * 10_000 > prev * maxPpsMoveBps) revert TooLargeMove();
            }
        }

        lastNav = NavSnapshot({ pps1e18: pps1e18, updatedAt: uint64(ts) });
        emit NavReported(pps1e18, ts);
    }
```

About Us

About Adevar Labs

Adevar Labs is a boutique blockchain security firm specializing in web3 audits.

Built by a mix of experienced professionals in traditional enterprise and crypto natives who have contributed to some of the most critical projects in blockchain infrastructure.

Our team s background spans companies like Bitdefender, Asymmetric Research, Quantstamp, Chainproof, and Juicebox, and includes experience securing smart contracts, bridges, and L1 and L2 protocols across ecosystems like Solana, Ethereum, Polkadot, Cosmos, and MultiversX.

With over 100 audits completed and a portfolio that includes custom fuzzers, exploit modeling, and runtime testing frameworks, Adevar Labs brings both depth and precision to every engagement.

Our auditors have discovered critical vulnerabilities, built high-impact tooling, and placed in top positions in premier audit competitions including Code4rena and Sherlock.

Team members hold distinctions such as PhDs in software protection, and key roles at Fortune 500 companies, and leadership of flagship conferences like ETH Bucharest.

With team members having publications with over 1,100 academic citations and trusted by projects securing over $500M in on-chain value, Adevar Labs blends elite technical rigor with real-world security impact.

We also collaborate with some of the best independent security researchers in the web3 space.

Projects may optionally request specific contributors to be part of their audit.

Audit Methodology

Our audit methodology is specialized to provide thorough security assessments of Ethereum smart contracts. We focus explicitly on rigorous manual analysis, detailed threat modeling, and careful validation of implemented fixes to ensure your programs operate securely and as intended.

1. Program Context and Architecture Analysis

Our auditors begin by deeply examining your Ethereum smart contract's documentation and intended functionality. We meticulously map out contract interactions, transaction processing flows, and state management logic. Special attention is given to understanding how your program interfaces with critical EVM standards, such as ERC-20, ERC-721, or ERC-4626 tokens, and governance/timelock modules. Last but not least we check external integrations with other DeFi protocols and verify if the inputs and return values are handled properly.

2. Threat Modeling

We conduct targeted threat modeling tailored specifically for EVM s execution environment. We carefully define attacker capabilities and identify potential vulnerabilities that may arise from EVM-specific issues, including but not limited to:

- Unauthorized tate variable modification
- Improper Access Control, role checks (onlyOwner, require(hasRole)) or msg.sender verification

- Misuse of low-level calls (call, delegatecall) or proxy/upgradeability patterns (e.g., storage collisions)
- Incorrect use of external calls, including reentrancy vectors
- Risks stemming from proxy initialization logic or denial-of-service related to gas limitations

3. In-depth Manual Security Review

Our experienced auditors perform an extensive manual security review of your Solidity-based EVM programs. This involves a comprehensive line-by-line inspection of source code, focusing on common EVM vulnerabilities including, but not limited to:

- Missing or insufficient Access Control and Role Checks
- Inadequate msg.sender and transaction origin checks
- Incorrect handling of external calls and invocation privileges, particularly potential reentrancy risks
- Arithmetic and integer overflow or underflow errors
- Unsafe use of ABI Encoding/Decoding or low-level assembly
- Improper ERC-20/Token Handling (e.g., approval logic, flash loan vectors, token locking)
- Logic flaws in financial operations or state transitions
- Edge-case handling in function input validation
- Potential denial-of-service vectors related to gas usage and transaction execution

During this stage, we clearly document any discovered vulnerabilities, including detailed descriptions, precise severity ratings, and recommendations for secure implementation. In situations where it is not clear how the vulnerability might be exploited we may also include a detailed proof-of-concept exploit including code snippets and instructions on how the exploit could be performed.

4. Detailed Fix Review and Validation

After the initial audit and your team s subsequent remediation efforts, we perform a comprehensive fix review to ensure the vulnerabilities identified have been effectively resolved.

We verify each fix individually, confirming that:

- Corrections effectively eliminate the security risks
- Changes do not inadvertently introduce new vulnerabilities or regressions
- The fixes align closely with EVM best practices and secure coding guidelines

Our detailed validation ensures that security improvements are robust, complete, and aligned with best practices specific to the EVM development ecosystem.

Confidentiality Notice

This report, including its content, data, and underlying methodologies, is subject to the confidentiality and feedback provisions in your agreement with Adevar Labs. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Adevar Labs.

Legal Disclaimer

The review and this report are provided by Adevar Labs on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Adevar Labs disclaims all warranties, expressed or implied, in connection with this report, its content, and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

You agree that access to and/or use of the report and other results of the review, including any associated services, products, protocols, platforms, content, and materials, will be at your sole risk.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided.

You acknowledge that blockchain technology remains under development and is subject to unknown risks and flaws. Adevar Labs does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open-source or third-party software, code, libraries, materials, or information accessible through the report. As with the purchase or use of a product or service in any environment, you should use your best judgment and exercise caution where appropriate.